

# QB: Chapter 3.3 — Hashing & File Organization

Unit 3 |  $5 \times 2$ -mark |  $5 \times 5$ -mark |  $4 \times 10$ -mark | **Total: 14 questions**  
Lectures: L42–L45 | BT Levels: BT1, BT2, BT3, BT5, BT6 | COs: CO2, CO3, CO4, CO5

---

## Section A: 2-Mark Questions

**Q1.** [BT1 | CO2] Define *hashing* and a *hash function*. State two properties that a good hash function must satisfy. Give an example of the division method hash function for integer keys.

---

**Q2.** [BT1 | CO2] Define a *collision* in a hash table. Name two collision resolution strategies and state whether each uses extra linked nodes or probes within the same array.

---

**Q3.** [BT2 | CO2] Define the *load factor*  $\alpha$  of a hash table. Write the formula relating  $\alpha$  to the number of stored keys  $n$  and table size  $m$ . Explain how a high load factor degrades the average lookup time under separate chaining.

---

**Q4.** [BT2 | CO4] Describe *sequential file organization*. State its primary advantage (simple structure) and its primary disadvantage (search efficiency). How does *indexed sequential* (ISAM) file organization improve on sequential access?

---

**Q5.** [BT3 | CO2] Given hash function  $h(k) = k \bmod 7$  and keys  $\{15, 11, 27, 8, 12, 5\}$ : Map each key to its slot. Identify all collisions (keys mapping to the same slot). State the resulting load factor.

---

## Section B: 5-Mark Questions

**Q6.** [BT3 | CO2] Apply *separate chaining* collision resolution to a hash table of size 7. Insert keys  $\{10, 22, 31, 4, 15, 28\}$  using  $h(k) = k \bmod 7$ . Draw the final hash table showing all chains. State the load factor and the length of the longest chain. Compute the expected number of comparisons for a successful search assuming uniform hashing.

---

**Q7.** [BT5 | CO2, CO3] Apply *linear probing* to a hash table of size 7. Insert keys  $\{50, 70, 76, 93, 85, 52\}$  using  $h(k) = k \bmod 7$ . For each key that causes a collision, show the complete probe sequence until an empty slot is found. Then, demonstrate the search for key 76: show each slot checked and the number of probes required.

---

**Q8.** [BT3 | CO2] Implement a *polynomial rolling hash* for string keys using the formula:  $h(s) = (s[0] \times \text{base}^{(n-1)} + s[1] \times \text{base}^{(n-2)} + \dots + s[n-1]) \bmod m$ , with  $\text{base} = 31$ ,  $m = 101$ .

(a) Compute  $h(\text{"cat"})$ ,  $h(\text{"act"})$ , and  $h(\text{"tac"})$ . (b) State whether anagrams always produce the same hash value with this function. Justify with the results from (a). (c) Explain how a better hash function design (e.g., sorted character signature) avoids anagram collisions.

---

**Q9.** [BT5 | CO2, CO3] Compare three open addressing strategies for a hash table of size 11 with primary hash  $h(k) = k \bmod 11$ : (a) linear probing, (b) quadratic probing ( $h(k, i) = (h(k) + i^2) \bmod 11$ ), (c) double hashing ( $h(k, i) = (h_1(k) + i \times h_2(k)) \bmod 11$ , where  $h_2(k) = 7 - (k \bmod 7)$ ).

Insert keys {20, 31, 42} using each scheme. Show the probe sequence for key 42 (which collides) under all three strategies. State which strategy avoids primary clustering and which avoids secondary clustering.

---

**Q10.** [BT3 | CO2] Build a character frequency table for the string "mississippi" using a hash map.

(a) For each distinct character, compute  $h(\text{char}) = \text{ord}(\text{char}) \bmod 10$  and show its slot. (b) Identify any slot collisions among characters. (c) Present the final frequency count for each character: m = 2, i = 4, s = 4, p = 2. Explain how the hash map handles the case where a character key already exists (update vs insert).

---

## Section C: 10-Mark Questions

**Q11.** [BT5 | CO2, CO3] (*Experiment E3 — Group Anagrams, LeetCode #49*)

The Group Anagrams problem groups a list of strings such that all anagrams of each other appear in the same group.

(a) Design a hash-map solution using the sorted version of each string as its key. Write the complete Python implementation. State the time complexity in terms of  $n$  (number of strings) and  $k$  (maximum string length).

(b) Trace the algorithm on input = ["eat", "tea", "tan", "ate", "nat", "bat"]. For each string, show the key computed (sorted string), the hash map state after insertion, and the final grouped output.

(c) Design an alternative key using a character frequency tuple (e.g., (1,0,0,...,1,...) for 26 letters). Write the key-generation logic for this approach. Compare the two approaches — sorted string vs frequency tuple — in terms of time complexity for key generation, hash collision frequency, and support for non-ASCII (Unicode) strings.

---

**Q12.** [BT5 | CO2, CO3] (*Experiment E14 — Valid Anagram, LeetCode #242*)

The Valid Anagram problem determines whether string  $t$  is an anagram of string  $s$ .

(a) Implement three approaches: (i) Sort both strings and compare —  $O(n \log n)$  time. (ii) Character frequency hash map (Python dict) —  $O(n)$  time. (iii) Fixed-size count array of 26 integers for lowercase English letters —  $O(n)$  time. Write Python implementations for all three.

(b) Compare the three approaches across: time complexity, space complexity, and whether

they support Unicode characters beyond ASCII. Recommend the best approach for: (i) strictly lowercase ASCII input, (ii) arbitrary Unicode input.

(c) Explain how the choice of collision resolution strategy in the underlying hash map affects the worst-case performance of approach (ii). Show that with a well-designed hash function and a load factor maintained at  $\alpha \leq 0.75$ , expected  $O(n)$  lookup performance is achieved. State what triggers a rehash operation and its amortised cost.

---

**Q13.** [BT6 | CO2, CO4]

A student database system must choose between four file organisation methods: sequential, relative (direct), index sequential (ISAM), and inverted file.

(a) Describe each file organisation method. For each, state: the access mechanism (by key or by position), whether records are physically sorted, and the primary intended use case.

(b) The database stores 10,000 student records sorted by StudentID. Using Big-O notation, compare sequential and index sequential (ISAM) access for each of the following operations: (i) search by StudentID (key field), (ii) search by Name (non-key field), (iii) insert a new record, (iv) delete an existing record. Justify each complexity.

(c) Design an *inverted file index* to support the combined query: "find all CS department students with CGPA > 8.0". Draw the index structure showing the inverted lists for Department and CGPA\_band (e.g., 8.0–9.0, 9.0–10.0). Describe the query resolution process (intersection of posting lists) and state its time complexity.

---

**Q14.** [BT6 | CO4, CO5]

You are the system architect of a high-traffic e-commerce platform processing 50,000 product searches per second over a catalog of 10 million items.

(a) Identify the most appropriate data structure for each of the following platform features, justify with time complexity analysis: (i) Full-text product keyword search. (ii) Price range queries (e.g., products priced between ₹500 and ₹2000). (iii) Recently viewed products per user (last 10 items, fast add/remove at ends). (iv) Shopping cart per user (fast insert, delete, and total-cost computation).

(b) The platform requires "users who bought X also bought Y" recommendations. Describe a data structure and algorithm to support this feature efficiently. Analyse the space complexity (in terms of distinct product pairs) and the query time complexity.

(c) Evaluate the trade-off between a hash table ( $O(1)$  average lookup) and a B-tree index ( $O(\log n)$  lookup) for the product catalog. Given that the catalog requires both exact-key lookups and range queries (price range, rating range), recommend and justify the optimal indexing strategy. Identify the scenario under which the hash table's  $O(1)$  average becomes  $O(n)$  worst case and how to mitigate it.